

Can Multi-LLM Negotiation Improve Scheduling Decisions?

A Market View of Elastic GPU Job Scheduling

Lay Kim Seng

Cyberscience Center, Tohoku University
lay.kim.seng.q4@dc.tohoku.ac.jp

[TODO: co-authors / advisor]

Abstract—A reasoning LLM has been proposed as the decision-maker for cluster scheduling, and multi-agent structures — one model proposing, another contesting — as the way to make that decision reliable. We ask whether the negotiation earns its cost on GPU job scheduling, and we argue that answering it requires first fixing the setting, because the standard one cannot express the question. Replaying the MIT Supercloud GPU trace through a malleability-aware batch simulator, we show that a *rigid* replay — every job pinned to the size it historically used, as in almost all published evaluations — fixes the device-seconds a workload consumes, leaving schedulers to differ only in *when* work runs: five schedulers separate by at most 1.3 points of occupancy, only 0.2% of idle capacity is recoverable by any ordering rule, and no ordering effect survives multiplicity correction. There is nothing for a negotiation to win. Restoring elasticity creates a genuine decision — how large to make each job — and on 12 windows selected from workload statistics alone, stratified by offered load and priority mix, that decision moves mean waiting time by $7\times$ ($p < 10^{-4}$). It also exposes a trap for any agent optimising the usual objective: the rule maximising utilisation is the worst scheduler we measure on every service axis, and holds the highest occupancy in 12 of 12 windows. Deterministic sizing does not solve the decision either — an adaptive rule fails to beat simply honouring each job’s requested size ($p = 0.77$ on waiting; 10.2 points worse on SLA, $p = 0.035$) — which leaves it genuinely open, and therefore a fair test. **[TODO: headline: single vs debate vs the deterministic floor across the 12 windows; runs in flight. Retain from the authored exception suite: debate corrects a single model’s rulings at an identical call budget, 43/81 vs. 29/81, $p = 0.0007$.]**

Index Terms—GPU scheduling, large language models, resource allocation, multi-agent debate, cluster traces

I. INTRODUCTION

AI training and inference are the dominant consumers of GPU clusters, and they are *elastic*: a training job can run on more or fewer devices, trading device-hours against completion time. A scheduler on such a workload therefore makes two decisions per job, not one — *which* job to start, and *how large* to make it.

Almost every published evaluation erases the second decision. A trace records that a job used g devices for t seconds; it does not record what that job would have done on $2g$ or on $g/2$, because the counterfactual was never observed. The standard response is to replay the trace *rigidly*, pinning each job to its historical size. This is usually presented as a conservative simplification. We show it is a suppression, and

that it silently invalidates the efficiency claims most commonly made on top of it.

The argument is short. If no job can be resized, the device-seconds a workload consumes are fixed: every scheduler must run the same jobs for the same durations, so it can only move work earlier or later. Utilisation is then close to a conserved quantity, and a comparison of schedulers on it is close to a comparison of nothing. Restoring elasticity does not merely add a tuning knob; it makes utilisation controllable, and — as we show — makes maximising it actively harmful.

Our contributions, all on the MIT Supercloud GPU trace replayed through a malleability-aware batch simulator [7]:

- 1) **The rigid replay cannot answer an efficiency question** (§IV-A). With jobs pinned to their historical sizes, five schedulers on the same window differ by at most 1.3 points of occupancy, and the idle capacity any ordering rule can still recover — devices left free while a job that fitted was waiting — is 0.2%. No ordering effect on service quality survives multiplicity correction either.
- 2) **Made elastic, sizing dominates ordering** (§IV-B). On 12 pre-registered windows, stratified by offered load and high-priority fraction and selected before any scheduler was run, the sizing rule alone moves mean waiting time by $7\times$ ($+232,451 \pm 60,823$ s, $p < 10^{-4}$) — an effect larger than any ordering difference we or the prior literature report on this class of trace.
- 3) **Utilisation is an anti-metric** (§IV-C). The rule that maximises occupancy is the worst scheduler on every service axis: $2.6\times$ the device-hours, $7\times$ the waiting, $+53.9$ points of SLA violation. In 12 of 12 windows the highest-occupancy arm is the worst-service arm. A scheduler optimising the reported metric would choose it.
- 4) **Sizing under uncertainty is open, not solved** (§IV-D). The natural remedy — shrink jobs when the queue is deep — does not beat simply honouring each job’s requested size (mean wait $p = 0.77$, device-hours $p = 0.36$, and SLA violations 10.2 points worse, $p = 0.035$). A single contended window suggested otherwise; it did not replicate, which is why the pre-registered stratification matters.

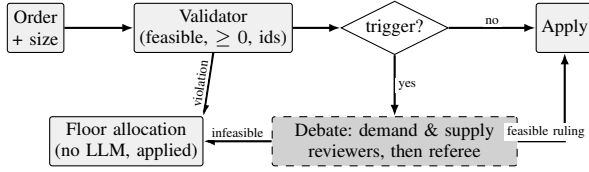


Fig. 1. The gated pipeline. Every scheduling point runs through deterministic code (grey): the ordering and sizing rules decide, the validator checks feasibility. Only a trigger (operator-note conflict, production arrival, deadline slip, contested-capacity boundary) escalates to the LLM debate (dashed). Any infeasible output, from either path, concedes the tick to the no-LLM floor; the LLM is never on the critical path.

We additionally retain, unchanged, a result that does not depend on any trace (§IV-E): on an authored suite of exceptions expressible only in operator text — which no public GPU trace supplies — a debate between LLM reviewers corrects a single model’s rulings at an identical call budget. That is where a reasoning layer earned its cost in our setting; it is not the numeric rationing decision, and §IV-C explains why the numeric decision has been measured against the wrong target.

II. RELATED WORK

LLM-driven scheduling. LLMsched [1] repairs a single LLM’s proposal with an ILP and reports large gains over Kubernetes. We inherit the reason-then-guarantee spine but invert its default: on numeric rationing the deterministic layer should be the proposer, not just the repairer, and the LLM should be gated behind a contextual trigger. **[TODO: 2–3 more LLM-scheduling refs]**

Learning-based schedulers. DRL schedulers (DeepRM [2], Decima [3]) learn allocation policies but are black-box and brittle under distribution shift. On our interface a tabular Q-learner’s episode-return variance dominates the action effect at realisable sample counts.

Multi-agent LLM structures. Multiagent debate [8] and self-consistency [9] are usually evaluated on QA benchmarks, and LLM-as-judge protocols [10] inherit that setting; budget-matched controls are rare, so a reported gain is rarely separable from the extra calls that buy it. Our text-exception result is budget-matched by construction and includes a blind confirmatory batch.

Malleability and mechanism design. Runtime elasticity is rarely deployed in production [4]; our partial-allocation model rests on it and we state that dependence. Uniform-price auctions are the classical route to truthful revelation, and automated negotiation for resource allocation is a classical subject [11], with prediction-guided *selective* negotiation explored in one-shot fair division [12]; our contribution is measurement in a closed loop containing declarative LLM agents.

III. THE GATED ARCHITECTURE

Fig. 1 shows the pipeline as evaluated.

Deterministic scheduling (default path, zero LLM). Every scheduling point runs two rules. An *ordering* rule sequences the pending queue; a *sizing* rule assigns each started

job a device count within its legal range. Rigid jobs admit exactly one size, so on a rigid replay the sizing rule is inert and the ordering rule is the entire policy — which is the situation §IV-A shows carries almost no signal. On an elastic workload both rules are live, and §IV-B shows the sizing rule dominates.

Validation (every scheduling point, cheap code). No proposal is applied as given. A deterministic check keeps only jobs that are actually pending, clamps each requested size into that job’s legal range and into the capacity still free, and drops the remainder in order. An unparseable or empty answer concedes the point to the no-LLM floor. This is feasibility enforcement, not repair: the mechanism never invents capacity, and it never silently rewrites a decision into a better one.

Escalation (only on a trigger). The reasoning path is consulted only when capacity must actually be rationed — when the work waiting exceeds what is free. Otherwise everything that fits is started and no call is made. When consulted, the model receives a textual packet (free capacity, and per job its identifier, legal size range, requested size, requested walltime, waiting time, priority tier, partition) and returns job/size pairs with a one-line justification; a *debate* variant adds a second reviewer that reads the first proposal before a final list is returned. The model never sees any job’s true runtime. Because the validator stands between the proposal and the cluster, an infeasible answer costs a fallback, not a violation, and reasoning is best-effort improvement rather than a step on the critical path.

IV. EVALUATION

Setup. We replay the MIT Supercloud GPU trace (a 235-day Slurm accounting log) through ElastiSim [7], a batch-system simulator for malleable workloads. One simulated node is one GPU, so a job that used g GPUs becomes a g -node job; its work is set so that it reproduces its recorded runtime exactly. The scheduler sees each job’s arrival, size and *requested* walltime, never its true runtime — the same information a production batch system has. Capacity is a fixed 80-GPU pool, so a window’s offered load is a property we *measure* rather than a consequence of sizing the cluster to fit it.

Window selection is pre-registered. Every 24 h window of the trace is described by workload statistics alone — job count, GPU-hours, offered load, high-priority fraction, median and p90 runtime, size distribution — with no scheduler executed. Of 5,613 hourly starts, 4,312 carry at least 100 jobs. These are stratified into four offered-load bands ($0.7\text{--}1.0$, $1.0\text{--}1.5$, $1.5\text{--}2.5$, $>2.5\times$) crossed with three high-priority bands ($<2\%$, $2\text{--}10\%$, $>10\%$), and one window is drawn per cell under a fixed seed, subject to a non-overlap constraint. The resulting 12 windows span days 41–222, offered loads $0.75\text{--}4.97\times$ and priority fractions $0.7\text{--}58\%$. Each is preceded by 12 h of warm-up arrivals, which the arm under test also schedules but which are excluded from scoring, so no run begins on an empty cluster. Selection never sees a scheduler result.

Elasticity model. A job may be *moldable*: its size is chosen once, at launch, and fixed thereafter. Speed-up follows Amdahl’s law anchored on the observed point — a job seen

at g GPUs reproduces its recorded runtime exactly at g , so the rigid replay is this same model evaluated at the size the job really ran, and the parallel fraction s is swept rather than assumed ($s=1$ is rigid at every size, $s=0$ is linear speed-up). Results below use $s=0.7$, a deliberately conservative value: scaling is sublinear, so growing a job costs more device-time than it saves. We report SLA- k (turnaround exceeding $k \times$ true runtime) at $k \in \{2, 5, 10\}$, mean waiting time, device-hours actually spent, walltime kills, and occupancy over the measured window.

A. The rigid replay cannot answer an efficiency question

With every job pinned to its historical size, the device-seconds consumed are fixed by the window, not chosen by the scheduler. Empirically this leaves almost nothing to compete over. Across five schedulers (strict FCFS, first-fit, EASY backfilling, requested-walltime ordering, and a declared/undeclared split) occupancy differs by at most 1.3 points within a window and is identical to three decimal places in several. The idle capacity any ordering rule could still recover — devices free while a job that fitted was waiting — averages 0.2%, with a worst case of 1.3%.

Service quality is equally flat. Over 11 windows no arm separates from the EASY baseline after Holm correction across three SLA thresholds; the largest raw effect, -4.2 points of SLA-2, carries $p_{\text{Holm}} = 0.41$. Two properties of the trace explain the flatness and are worth recording, because both are usually assumed away. First, EASY backfilling is inert here: across 1,351 blocked scheduling points it backfilled 48 times (3.6%), because requested walltimes exceed true runtimes by 100–600 $\times$, so no job can be shown to fit before the head job’s reservation. EASY and strict FCFS are byte-identical on most windows. Second, requested walltime carries no runtime signal: among 487 jobs in one window there are 9 distinct requested values, and the rank correlation with true runtime is $+0.16$, -0.11 , -0.73 and -0.01 across four windows — the only significant value is negative. Any “shortest-job-first” arm on this trace is therefore ordering on noise.

B. Made elastic, sizing dominates ordering

We now allow every job to be moldable and hold the ordering rule fixed at first-fit, varying only how each job is sized: AS-REQUESTED honours the size its owner asked for and waits until that many devices are free; GREEDY takes the largest legal size that fits; ADAPTIVE shares free capacity across the waiting queue. Table I reports means over the 12 pre-registered windows.

The spread is an order of magnitude, on an axis that does not exist in the rigid replay. Paired against AS-REQUESTED, GREEDY costs $+5,929 \pm 1,707$ device-hours ($p=3 \times 10^{-5}$), $+232,451 \pm 60,823$ s of mean waiting ($p=10^{-5}$) and $+53.9 \pm 12.3$ points of SLA-10 ($p<10^{-5}$). No ordering rule we measured in §IV-A moves any metric by a comparable amount.

C. Utilisation is an anti-metric

The three sizing rules order identically on occupancy and inversely on service. GREEDY attains the *highest* occupancy

TABLE I
SIZING RULE, 12 PRE-REGISTERED WINDOWS, $s = 0.7$, FIRST-FIT
ORDERING HELD FIXED. MEAN $\pm$ 95% CI.

sizing	device-h	occupancy	SLA-10 (%)	mean wait (s)
AS-REQUESTED	3653	0.890	22.3	38,072
ADAPTIVE	3339	0.964	32.5	39,756
GREEDY	9581	0.988	76.2	270,522

of any arm we measure — 0.988, against 0.890 for AS-REQUESTED ($+0.1 \pm 0.1$, $p=0.009$) — while being the worst on every service axis simultaneously. In 12 of 12 windows the highest-occupancy arm is the worst-service arm, and in 10 of 12 the highest-occupancy arm is not the best-service arm.

The mechanism is not subtle, which is what makes the metric dangerous. Under sublinear scaling, a job given $4 \times$ the devices at $s=0.7$ finishes about $1.6 \times$ sooner while consuming $4 \times$ the capacity. GREEDY therefore converts the cluster into *busy-ness* without converting it into completed work, and the queue behind it grows. Occupancy records the busy-ness and is blind to the waste: on one low-load window it reads 1.000 for all three sizing rules while their mean waiting times span 21,429 s to 244,781 s.

This is the practical consequence of §IV-A. A rigid replay reports occupancy differences of a fraction of a point and no scheduler can game them; the metric looks harmless because it is inert. Made elastic, the same metric becomes controllable — and the controller that maximises it is the one to avoid. We therefore recommend reporting device-hours spent per unit of completed work, together with waiting time, and retiring occupancy as a headline number for elastic workloads.

D. Sizing under uncertainty is open, not solved

The obvious fix for GREEDY is to shrink jobs when the queue is deep. It does not work. Paired over the 12 windows, ADAPTIVE does not beat AS-REQUESTED on mean waiting ($+1,684 \pm 10,847$ s, $p=0.77$) or on device-hours (-313 ± 637 , $p=0.36$), and is significantly *worse* on SLA-10 ($+10.2 \pm 8.3$ points, $p=0.035$). It also introduces a failure the other rules do not have: shrinking a job lengthens it, and at lower parallel fractions this pushes long jobs past their requested walltime and gets them killed. We therefore count kills as SLA violations rather than as fast completions, since a killed job otherwise flatters every wait-based metric.

We record one negative result about our own method. On the single most contended window ($4.97 \times$ offered load), ADAPTIVE cut mean waiting by 42% against AS-REQUESTED and looked like a clear win. It did not replicate across the pre-registered set. Had the window been chosen after seeing that number, the effect would have been reported as a finding. This is the concrete argument for selecting windows from workload statistics alone.

What remains open is therefore the interesting decision: sizing a job under genuine uncertainty about its runtime, on a trace where the only available estimate is uninformative (§IV-A). We regard that, not ordering, as the decision on which

TABLE II

THE TEXT-EXCEPTION SUITE: 81 PRIMARY + 17 CONTROL CASES.

category	n	the exceptional fact arrives as...
nl_policy	21	an operator rule that exists only in words
unmodeled	21	a fact the world model does not represent
corrupt	13	a declaration a note says is wrong
contradiction	10	two statements that cannot both hold
ambiguous	8	an under-specified scene needing judgement
infeasible	8	demand that cannot be met as declared
placebo	9	decorative/expired prose (control)
confirm	8	prose confirming the default (control)

a reasoning layer should next be asked to earn its cost [TODO: fold in the LLM sizing arms across the 12 windows; the current LLM result is a single window and directional only].

E. Where reasoning earns its cost: text exceptions

A deterministic mechanism cannot act on an exception whose content is *text*: an operator instruction, a declaration a note says is wrong, a crash-restart the numbers do not show. We measure this on a pre-registered suite of hand-authored exception scenes.

The suite (Table II). Each case is a small allocation scene (median two jobs, one supply note, 2–8 free GPUs, ~36 words of prose) with a pre-registered predicate defining the defensible ruling, authored *before* any LLM was run on it. The authoring rule is text-dependence: deleting the exception sentence leaves a scene whose numbers imply a different, perfectly reasonable answer. **Admission required both rigid arms — a fixed ILP over the same submissions and a rule-based floor — to fail the predicate; the 0/81 rigid floor in Table III is therefore an inclusion criterion, not a finding.** What the suite measures is the comparison *among* arms that can read text. Two control categories guard the boundary: *placebo* scenes with decorative or expired prose (including a keyword trap naming a policy the note itself retracts) and *confirm* scenes whose salient prose confirms the default ranking; the fixed-ILP arm and the deterministic market pass 17/17 and 16/17 of these, respectively. Of the 81 primary cases, 50 were authored blind to the per-case outcomes of the earlier under-powered round, and the harness reproduces that round’s head-to-head exactly ($b=6$, $c=1$).

Arms, budget-matched by construction. All arms read an identical structured decision packet. Alongside a single LLM call we run best-of- N sampling given the *same* call budget as the debate, so the only difference between the two expensive arms is how the budget is spent, not how much there is.

The pre-registered primary contrast, debate vs. budget-matched best-of- N : $b=16$, $c=2$, one-sided exact McNemar $p = 0.0007$. Three controls make this hard to explain away: (i) *budget alone buys nothing* — best-of- N spends $7\times$ the single arm’s calls and gains two cases ($p = 0.25$, ns), while debate spends the identical budget and gains sixteen; (ii) *a blind confirmatory batch reproduces it* — on the 50 cases authored blind to earlier outcomes, debate 29/50 vs. 19/50,

TABLE III

HANDLED CASES ON THE 81 PRIMARY TEXT EXCEPTIONS (QWEN2.5:14B).

arm	handled	LLM calls
rigid floor (inclusion criterion)	0/81	0
single LLM	27/81	81
single LLM, best-of- N (budget-matched)	29/81	569
debate	43/81	569

$b=11$, $c=1$, $p = 0.0032$; (iii) *the effect respects the boundary* — on the 17 control scenes every LLM arm scores 12/17 against the deterministic market’s 16/17, i.e. reasoning invoked where there is nothing to reason about is actively harmful. An exploratory per-category split (post-hoc, small cells) locates the gain where a fact is *missing* from the model of the world (unmodeled: 13/21 vs. 4/21, $b=9$, $c=0$) rather than where an instruction is present (nl_policy: 8/21 in every LLM arm): debate helps when reviewers must notice an absence, not follow a presence. [TODO: if the gemma2:9b/27b replication lands: one sentence on cross-family robustness] [TODO: rater-agreement number for the predicates, threats para]

F. Why the suite must be authored

The natural objection is that an authored suite proves little about a real workload. It is the only option available. The Supercloud accounting log carries 29 fields — identifiers, timestamps, requested and allocated resources, partition, priority, exit state — and not one of them is free text. No public GPU or HPC trace we surveyed — including the other widely used GPU trace [5] — ships operator-authored exception text; what text exists in this domain is source code and logs, not scheduling rationale.

This is the boundary, and it cuts both ways. A reasoning layer cannot demonstrate value on a channel the data does not contain, which is why §IV-E is measured on an authored suite rather than in simulation. It also means the numeric results of §IV-A–§IV-D are the whole of what this trace can adjudicate, and on those the decisive quantity is a sizing rule, not a reasoning step. Acquiring a workload that carries operator text remains the prerequisite for testing the exception claim in situ.

V. THREATS TO VALIDITY

The elasticity model is an assumption, and a swept one. No trace records what a job would have done at a different size, so any elastic study must supply that counterfactual. We supply Amdahl’s law anchored on the observed point, which reproduces each job’s recorded runtime exactly at the size it really used, and we sweep the parallel fraction rather than fixing it: $s=1$ recovers the rigid replay at every size and $s=0$ gives linear speed-up. Our conclusions concern the *shape* of the result across that sweep, not a single value. Real training jobs also pay a reconfiguration cost we do not model, which would penalise GREEDY further; our claim that occupancy-maximising sizing is harmful is therefore conservative.

One node is one GPU. Supercloud’s TX-GAIA nodes carry two V100s; we model the allocation unit as a device, so intra-node placement effects are absent.

Moldable, not malleable. Sizes are chosen once at launch and fixed. Mid-run resizing — elastic training in the strict sense — would give a scheduler strictly more power, and we do not measure it.

SLA- k is an oracle metric and is length-biased. It is defined from a job’s true runtime, which no scheduler knows at submission, so no arm can target it; and because the allowance scales with the job, short jobs violate readily while the longest decile of jobs cannot violate at all within a 24 h window despite waiting longest. We therefore report $k \in \{2, 5, 10\}$ throughout and lead with waiting time and device-hours, which carry neither defect. The trace supplies no deadline field, so no absolute deadline claim is available.

Twelve windows, one per stratum. A difference confined to one cell cannot be separated from that particular window; the strata establish that a result is not confined to one corner of the load/priority space, not that a stratum causes it. §IV-D reports a case where this mattered.

Only completed GPU jobs are replayed, so failed and cancelled work, non-GPU contention, and the cluster’s background load are absent. LLM latency is hidden behind wait time by the gate in the design, but evaluation runs on the simulation clock.

VI. CONCLUSION

Elastic workloads make a scheduler choose job *size*, and that decision dominates the ordering decisions the literature usually measures — by an order of magnitude on our pre-registered windows. It also breaks the metric the literature usually reports. Because sublinear scaling lets a scheduler spend four times the capacity to finish a job 1.6 times sooner, the rule that maximises utilisation is the worst scheduler we measure, and it is the worst in every one of twelve windows. Utilisation records busy-ness; on an elastic workload busy-ness and completed work come apart, and optimising the former destroys the latter.

The rigid replay hides all of this. With sizes pinned, device-seconds are conserved, occupancy varies by fractions of a point, and the metric appears both stable and harmless — which is precisely why efficiency claims made in that setting should not be read as claims about elastic clusters. The instrument cannot detect the failure it is being used to rule out.

We do not offer a solved replacement. The natural adaptive rule fails to beat simply honouring what users asked for, and a promising result on the single most contended window did not survive the pre-registered set. Sizing under uncertainty — on a trace whose only runtime estimate is uncorrelated with truth — is where we think the next contribution lies, and it is a decision on which a reasoning layer has yet to be fairly tested. Where reasoning did earn its cost in our setting was on exceptions expressible only in operator text, a channel no public GPU trace currently supplies.

REFERENCES

- [1] [TODO: LLM-Driven Adaptive Cloud Resource Scheduling: Bridging Reasoning Intelligence with Optimization Guarantees. *IEEE OJ-CS, 2026* — exact citation]
- [2] H. Mao et al., “Resource management with deep reinforcement learning,” *HotNets*, 2016.
- [3] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” *SIGCOMM*, 2019.
- [4] A. Tarraf et al., “Malleability in modern HPC systems: current experiences, challenges, and future opportunities,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 9, pp. 1551–1564, 2024.
- [5] Q. Weng et al., “MLaaS in the wild: workload analysis and scheduling in large-scale heterogeneous GPU clusters,” *NSDI*, 2022 (cluster-trace-gpu-v2020).
- [6] D. Lifka, “The ANL/IBM SP scheduling system,” *JSSPP*, 1995.
- [7] T. Özden, T. Beringer, A. Mazaheri, H. M. Fard, and F. Wolf, “ElastiSim: a batch-system simulator for malleable workloads,” *ICPP*, 2022, pp. 1–11.
- [8] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” *ICML*, 2024, pp. 11733–11763.
- [9] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” *ICLR*, 2023.
- [10] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” *NeurIPS Datasets and Benchmarks Track*, 2023.
- [11] B. An, “Automated negotiation for complex multi-agent resource allocation,” Ph.D. dissertation, Univ. of Massachusetts Amherst, 2011.
- [12] M. Croitoru, C. Croitoru, and G. Ganesh, “Prediction-based selective negotiation for refining multi-agent resource allocation,” *ICAART*, vol. 1, 2025, pp. 656–662.